

WRO Future Engineers 2026 Season

Team C.O.D.E Documentation

Cognitive Operations & Digital Engineering

Jadon Steele & Mischa Miller

HelderBerg Robotics Club

Index

1 Introduction

1.1 Team Introduction

1.2 Robot Introduction

2 Robot Hardware

2.1 Interfaces

2.1.1 UART

2.1.2 I2C

2.2 Electronics

2.2.1 RDK X5

- Libraries

- Reason

2.2.2 Power

- Batteries

- Blade 5A Fuse & Fuse Holder

- Buck Converter

2.2.3 Sensors

- Huskylens V1

- Gyro (DFRobot 6-Axis Accelerometer)

- Capacitive Touch Sensor

2.2.4 Movement

- DS-R001 – Clutch Servo Motor

- DC Motor

- L298N

2.2.5 Wiring Diagram

2.3 Chassis/Other

3 Software and Strategy

3.1 Open challenge Strategy

3.2 Obstacle challenge Strategy

3.3 Calibration

3.4 Open challenge flowchart

3.5 Obstacle challenge flowchart

4 Robot Journey

4.1 Changes from 2025 Season

4.2 Monthly Logs

- January Log**
- February Log**
- March Log**
- April Log**
- May Log**
- June Log**
- July Log**
- August Log**

Team Introduction: Team C.O.D.E

We are Team C.O.D.E, a two-person team competing in the WRO Future Engineers category. We first met during last season's FTC competition and quickly discovered that we shared a passion for robotics, engineering, and programming. Since then, we've worked together to design, build, and improve an autonomous vehicle that can tackle the challenges of the Future Engineers competition.

Meet the Team:

Mischa Miller

Role: Strategy & Quality Assurance Lead

Mischa is responsible for planning, testing, and keeping the project organised. He manages our engineering journal, tracks our progress, and helps make sure every change we make is tested and documented. His focus is on making sure the team stays on schedule and that our robot performs consistently and reliably.



Jadon Steele

Role: Lead Engineer & Programmer

Jadon leads the design and development of our robot. He is responsible for programming the autonomous system, integrating sensors, and improving the mechanical design. Drawing on several years of robotics competition experience, he enjoys solving technical problems and finding ways to make our robot more accurate and reliable.



Our Goal:

Our goal is to represent South Africa for the WRO International Finals. To get there, we're focused on building a reliable autonomous vehicle that can consistently complete every challenge in the Future Engineers competition. We know that success comes through careful planning, regular testing, and continuously improving our design. No matter the result, we hope to learn new skills, gain valuable experience, and represent our team to the best of our ability.

Robot Introduction

Meet the WRO FE MSE-6 (also known as Mouse), our autonomous vehicle designed specifically for the WRO Future Engineers competition. Every component was selected to maximise reliability, performance, and consistency during autonomous operation.

The robot uses a tri-camera HuskyLens AI setup. Two cameras face forward, each with a 60° field of view, giving wide coverage of the competition field. A third camera faces downward toward the mat, replacing the original colour sensor.

- Forward cameras:
 - Detect lane markings, pillars, and obstacles.
 - Provide a combined forward view that improves navigation accuracy and reduces blind spots.
- Downward camera:
 - Captures mat colours and features with greater precision than the previous colour sensor.
 - Operates at higher speed, enabling faster runs without sacrificing detection reliability.
 - Supplies data for mapping and route planning, feeding into the robot's ID set → route set pipeline.

Advantages of the Tri-Camera System

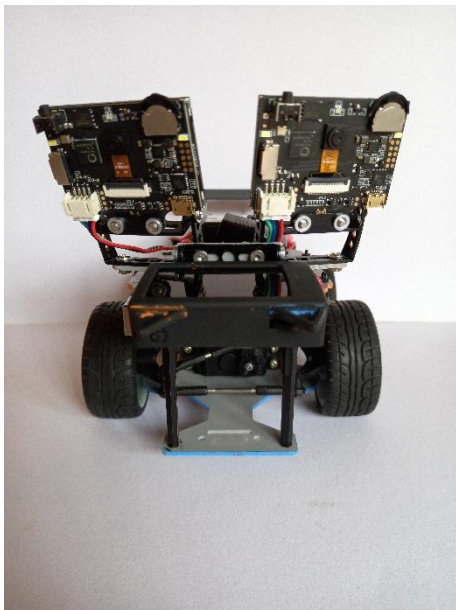
- Higher accuracy: Downward HuskyLens delivers richer colour and pattern recognition.
- Speed optimisation: Robot maintains faster lap times while keeping detection consistent.
- Consistency: Reduces twitching and misreads, ensuring smoother lap execution.

Processing the large amount of data from the cameras requires a powerful onboard computer. For this reason, we chose the RDK X5, which features an octa-core ARM Cortex-A55 processor and 8 GB of LPDDR4 memory. This provides enough computing power to run our vision algorithms and control systems in real time, allowing the robot to operate fully autonomously.

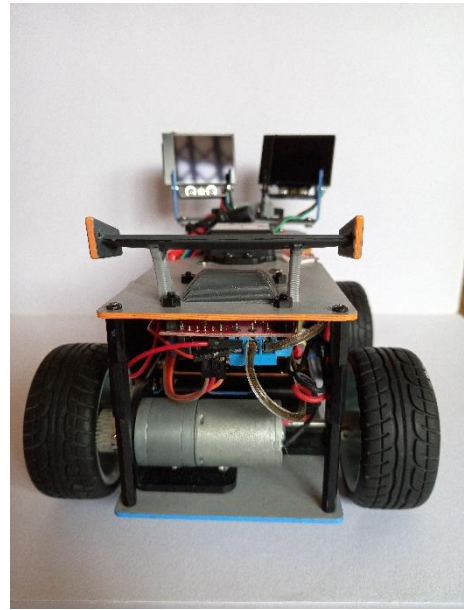
The following sections describe the robot's mechanical design, electronics, software, and the testing process that led to the final version of the MSE-6.

For the Mechanical, electronic, and software design we tried to keep it as simple as possible

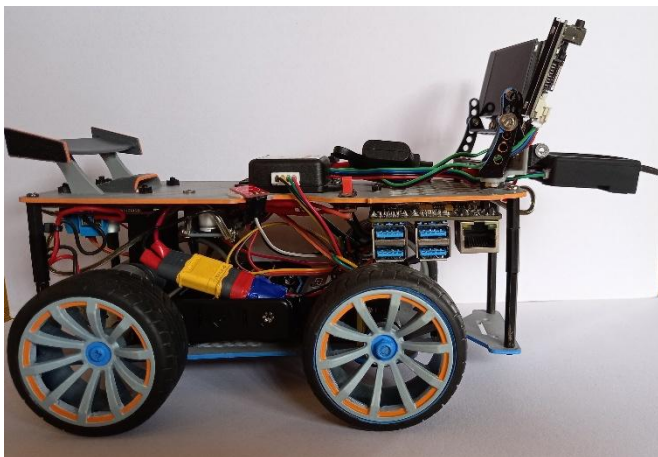
Robot Photos:



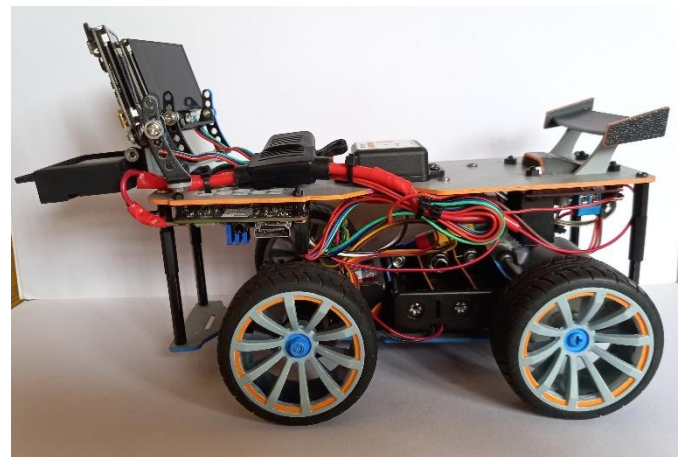
[Front]



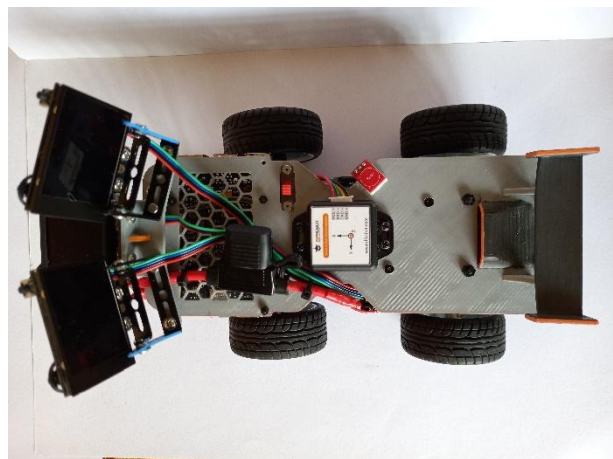
[Back]



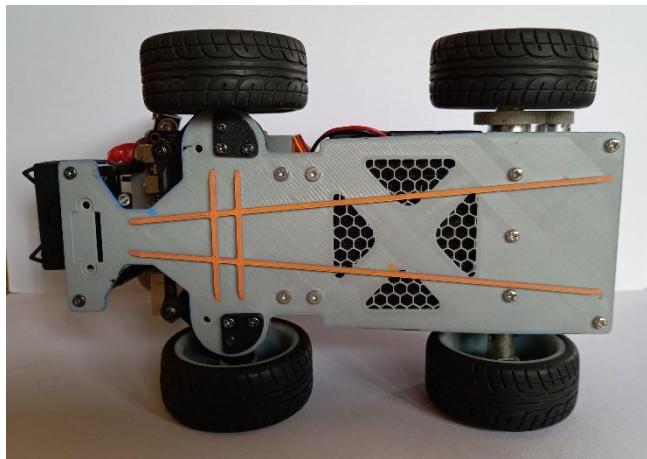
[Right]



[Left]



[Top]



[Bottom]

Robot Hardware/Specifications:

Interfaces:

UART:

Universal Asynchronous Receiver/Transmitter (UART)

Definition:

UART is a hardware communication protocol that enables asynchronous serial data transmission between devices. It converts parallel data from a microcontroller or computer into serial form for transmission and reconverts received serial data back into parallel form.

Key Characteristics:

- **Asynchronous Communication:** No shared clock signal; synchronization is achieved using start and stop bits.
- **Full Duplex:** Supports simultaneous transmission (TX) and reception (RX).
- **Frame Structure:** Data is sent in packets consisting of:
 - Start bit (signals beginning of data frame)
 - Data bits (usually 7, 8, or 9 bits)
 - Optional parity bit (error detection)
 - Stop bit(s) (signals end of frame)

Typical Parameters:

- **Baud Rate:** Speed of transmission (e.g., 9600, 115200 bps).
- **Data Bits:** Commonly 8 bits per frame.
- **Parity:** None, even, or odd.
- **Stop Bits:** 1 or 2 bits.

Applications:

- Microcontroller ↔ Sensor communication (e.g., accelerometers, GPS modules).
- Debugging and logging via serial consoles.
- Embedded systems interfacing (Arduino, Raspberry Pi, etc.).
- Peripheral connections (Bluetooth modules, GSM modems).

Hardware Interface:

- **TX (Transmit):** Sends data out.
- **RX (Receive):** Reads incoming data.
- **GND:** Common ground reference.
- Optional **RTS/CTS** lines for hardware flow control.

Advantages:

- Simple and widely supported.
- Requires only two data lines (TX, RX).
- Flexible baud rate configuration.

Limitations:

- Point-to-point only (not multi-drop like RS-485).
- Limited distance and speed compared to modern protocols (e.g., USB, SPI).
- No inherent clock synchronization — requires precise baud rate matching.

Components Using UART:

- 3x HUSKYLENS V1
 - Gyro
-

I2C:

Inter-Integrated Circuit (I²C)

Definition:

I²C is a synchronous, multi-master, multi-slave, serial communication protocol developed by Philips (now NXP). It enables short-distance communication between integrated circuits using only two bidirectional lines: SDA (Serial Data) and SCL (Serial Clock).

Key Characteristics:

- Synchronous Communication: Data transfer is synchronized with a shared clock (SCL).
- Two-Wire Interface: Only SDA and SCL lines are required, plus ground.
- Multi-Device Support: Multiple masters and slaves can share the same bus.
- Addressing: Each slave device has a unique 7-bit or 10-bit address.
- Open-Drain Design: Devices pull the line low; external pull-up resistors are required.

Typical Parameters:

- Clock Speed: Standard (100 kHz), Fast (400 kHz), Fast Plus (1 MHz), High-Speed (3.4 MHz).
- Data Frame Structure: • Start condition (SDA pulled low while SCL is high) • Address frame (7/10 bits + R/W bit) • ACK/NACK bit (acknowledgment from receiver) • Data frames (8 bits each, followed by ACK/NACK) • Stop condition (SDA released high while SCL is high)

Applications:

- Communication between microcontrollers and peripherals (EEPROMs, sensors, displays).
- Configuration of ICs in embedded systems.
- Power management ICs and real-time clocks.
- Consumer electronics (smartphones, TVs, cameras).

Hardware Interface:

- SDA (Serial Data): Transfers data between devices.
- SCL (Serial Clock): Synchronizes data transfer.
- Pull-Up Resistors: Required on both lines to maintain logic high when idle.

Advantages:

- Simple wiring (two lines for many devices).
- Supports multiple masters and slaves.
- Well-suited for low-speed, short-distance communication.

Limitations:

- Limited speed compared to SPI or UART.
- Bus capacitance restricts maximum distance and number of devices.
- Requires careful pull-up resistor sizing for reliable operation.

Components Using I²C:

- TCS34725 – Colour Sensor (Removed)
-

Electronics:

RDK X5:

The **RDK X5** was chosen as the main controller for the robot because it offers the best balance of **AI acceleration, connectivity, and GPIO support** compared to other SBCs in the comparison:

AI Acceleration:

- Equipped with a **10 TOPS BPU (Neural Processing Unit)**, the RDK X5 is far more capable for edge AI and robotics tasks than the Raspberry Pi 5 (limited AI acceleration) and Jetson Nano (fixed 4 GB RAM, older GPU).

Connectivity:

- Built-in **Wi-Fi 6 + Bluetooth 5.4** ensures modern wireless standards, unlike the Jetson Nano which lacks onboard Wi-Fi.

GPIO & Interfaces:

- Provides **28 GPIOs** with support for **SPI, I²C, I²S, PWM, and UART**, covering all the communication protocols needed for sensors and actuators in the robot.

Form Factor:

- Compact dimensions (**85 × 56 × 20 mm**) make it stackable and ideal for embedded robotics, unlike the larger Jetson Nano or LattePanda Delta 3.

Power Input:

- Runs on a simple **5V/5A USB-C supply**, compatible with the robot's buck converter and battery system.

OS Support:

- Supports **Ubuntu 22.04 and RDK OS Linux**, giving flexibility for robotics frameworks like ROS2.

Why Not the Others?

- **Raspberry Pi 5:** Excellent community support and dual 4K display, but limited AI acceleration and requires active cooling.
- **Jetson Nano:** Strong AI features with CUDA, but restricted by fixed 4 GB RAM and no onboard Wi-Fi.
- **LattePanda Delta 3:** Runs Windows/Linux with powerful x86 CPU, but higher power draw and larger form factor make it less suited for compact robotics.

Conclusion:

The **RDK X5** was selected because it combines **AI performance, modern connectivity, versatile GPIO support, and compact design**, making it the most suitable SBC for an **edge AI robotics platform**.

Power:

Battery System – 18650 Lithium Cells (3S1P Pack)

Configuration

- Type: 18650 Lithium-ion cells
- Arrangement: 3S1P (3 cells in series, 1 parallel string)
- Nominal Voltage per Cell: 3.7 V
- Capacity per Cell: ~3000 mAh (3.0 Ah)

Pack Specifications

- Nominal Voltage: $V_{total} = 3 \times 3.7 \text{ V} = 11.1 \text{ V}$
- Maximum Voltage (Fully Charged): $V_{max} = 3 \times 4.2 \text{ V} = 12.6 \text{ V}$
- Total Capacity: $C_{total} = 3.0 \text{ Ah}$
- Energy Stored: $E = 11.1 \text{ V} \times 3.0 \text{ Ah} \approx 33.3 \text{ Wh}$

Current Output

- Continuous Current: Depends on the discharge rating of the cells. • Standard 18650 cells: ~5–10 A continuous • High-drain 18650 cells (used in power tools, e-bikes, robotics): 15–20 A continuous, sometimes higher
- Peak Current: Short bursts can exceed 20 A depending on cell quality.
- Pack Limitation: Since this is a 3S1P pack, the current capability is limited to what a single cell can provide (no parallel strings to share load).

Practical Notes

- Runtime per pack: ~45 minutes under typical robot load.
- Multiple packs are rotated to ensure a fully charged set is always available.
- Inline 5 A fuse in combination with the Buck Converter is used for protection of the board, but the cells themselves can deliver much higher current.

Buck Converter – 5V/5A (XL4015)

Part Description

The XL4015 is used to step down our 11.1V battery to a stable 5V to safely power the RDK X5 and HuskyLenses. We specifically required a regulator with a 5A limit because our combined AI processing and vision system draws a peak of ~4.19A. We chose this switching buck converter over a linear regulator because of its high conversion efficiency (up to 96%), ensuring that power isn't wasted as heat during our 3-lap runs.

Quick Specifications

- Input Voltage: 4 – 38 V
- Output Voltage: 1.25 – 32 V (adjustable)
- Output Current: 5 A (max; recommended ≤ 4.5 A)
- Output Power: 75 W
- Working Temperature: –40 to +85 °C
- Working Frequency: 180 kHz
- Conversion Efficiency: up to 96% (depends on input/output voltage, current, and voltage drop)
- Load Regulation Rate: $S(I) \leq 0.8\%$
- Voltage Adjustment Rate: $S(U) \leq 0.8\%$
- Protection: Short-circuit protection, Over-temperature protection
- Dimensions: 54 × 23 × 18 mm
- Net Weight: 13 g

Fuse – 5A Blade Type

Part Description

The 5A blade fuse is a critical safety mechanism for our robot's logic circuit. We installed it directly *after* the XL4015 Buck Converter on the 5V line before the power reaches the RDK X5 and HuskyLenses. We explicitly chose a 5A rating because our peak logic current draw is calculated at ~4.19A. This provides enough headroom for AI processing spikes, but guarantees that if a short circuit occurs, the fuse will blow before our sensitive logic components melt or the buck converter exceeds its maximum rating.

Quick Specifications

- Rated Voltage: 32 V
- Rated Current: 5 A
- Fuse Colour: Clear Orange
- Dimensions: 19 × 19 × 5 mm
- Net Weight: 13 g

Fuse Holder – Inline Blade Type

Part Description

This inline holder integrates the 5A blade fuse into our main wiring harness. We chose an inline automotive-style holder because it provides a highly secure, vibration-proof connection during high-speed runs and sharp 45° turns. Furthermore, its simple plug-and-play design means that if we blow a fuse during a competition round, we can physically replace it in the pits in under 10 seconds without needing to solder or rewire the chassis.

Quick Specifications

- Max Voltage: 48 V
- Max Current Rating: 32 A (Peak 40 A)
- Fuse Seat Size: 40 × 35 × 24 mm
- Net Weight: 23 g

Sensors:

Huskylens V1 – AI Vision Sensor:

Part Description

The HuskyLens is the core of our robot's vision system. We transitioned from a 2025 sensor array (Time-of-Flight + traditional colour sensors) to a custom **Tri-Camera HuskyLens setup**. One camera faces downward for high-speed mat colour recognition (track mapping), while two face forward, angled to provide a massive 120° combined field of view to detect traffic pillars on corners. We chose the HuskyLens specifically to offload processing from our main computer. Instead of the RDK X5 analysing raw video pixels, the built-in Kendryte K210 AI chip on the HuskyLens handles the heavy object-recognition math, sending only lightweight coordinate data to the RDK X5 via I2C/UART. This brilliant division of labour drops our system latency and drastically improves lap times.

Quick Specifications

- Processor: Kendryte K210
 - Image Sensor: OV2640 (2.0 Megapixel Camera)
 - Supply Voltage: 3.3 – 5.0 V
 - Current Consumption (Typical): 320 mA @ 3.3 V, 230 mA @ 5.0 V (face recognition mode; 80% backlight brightness; fill light off)
 - Connection Interface: UART, I²C
 - Display: 2.0-inch IPS screen, 320 × 240 resolution
 - Built-in Algorithms: Face Recognition, Object Tracking, Object Recognition, Line Tracking, Colour Recognition, Tag Recognition
 - Dimensions: 52 × 44.5 mm
-

Gyro – DFRobot 6 axis accelerometer:

Part Description

The gyro is the cornerstone of our robot's straight-line navigation. In Round 1, the robot relies entirely on this sensor's heading data to hug the outer wall of the track at high speeds. We specifically selected this DFRobot module over cheaper alternatives (like the standard MPU6050) because it features an integrated microprocessor running advanced Kalman filter algorithms. This means the sensor internally cleans up its own noise and calculates an accurate attitude angle *before* sending data to the RDK X5. By offloading this math, we save valuable CPU cycles on the RDK X5 for processing our three camera streams. During testing, we applied a deliberate 10° software offset to the gyro's zero-heading, which biases the robot slightly inward during runs to prevent accidental outer-wall clipping.

Quick Spec

- Voltage: 3.3V~5V
 - Current: <40mA
 - Measuring Dimension: acceleration: 3D; angular velocity: 3D; attitude angle: 3D
 - Range: acceleration: $\pm 2/4/8/16g$ (optional); angular velocity: $\pm 250/500/1000/2000^\circ/s$ (Optional); attitude angle: $\pm 180^\circ$
 - Dimensions: 51.3 x 36 x 10mm
 - Net Weight: 18g
-

TTP223 Capacitive Touch Sensor:

Part Description

The WRO rulebook requires a strict, physical start procedure to initiate the autonomous run. We utilize the TTP223 as our main start trigger. We chose a capacitive touch sensor over a traditional mechanical push-button to solve a critical software issue: "signal bounce."

Mechanical switches physically bounce when pressed, sending dozens of rapid ON/OFF signals in a matter of milliseconds, which can accidentally multi-trigger or crash our software's bit-bashing loop. The TTP223 IC completely eliminates this issue, delivering a single, digitally clean, and debounced signal that guarantees a flawless launch off the starting line every single time.

Quick Spec

- IC: TTP223 Touch Pad Detector
 - Input Voltage: 2.0V – 5.5V DC
 - Standby Current: < 10 μ A
 - Response Time: ~60ms (Fast Mode) / ~220ms (Low Power Mode @3V)
 - Touch Detection Range: 0 – 10mm (depending on surface material)
 - Output Modes: Momentary or Latching / Active-High or Active-Low selectable
 - Sensitivity Adjustment: External capacitance (0–50pF)
 - Auto Calibration: Recalibrates every 4 seconds (no touch)
 - Operating Temperature: –40°C to +85°C
 - Dimensions: 15mm × 11mm × 2.5mm
-

Movement

DS-R001 – 6Kg Clutch Servo Motor:

Part Description

This servo acts as our front-wheel steering actuator. We upgraded to this clutch servo after our original MG996R burned out during testing. The built-in clutch mechanism is vital for our robot's reliability: if the robot crashes into a track wall, the clutch slips at 6kg of torque rather than stripping the internal gears or stressing our 3D-printed chassis linkage. This gives us much smoother and safer steering.

Quick Spec

- Operating Voltage: 4.8-6V DC
 - Static Current: $\leq 10\text{mA}$ @ 6.0V
 - Current (at no load): $\leq 60\text{mA}$ @6.0V
 - Current (at stopped): $\leq 1.65\text{A}$ @6.0V
 - Rated Torque: $\geq 4.4\text{kg}\cdot\text{cm}$ @6.0V
 - Stall Torque: $6\text{kg}\cdot\text{cm}$ @6.0V
 - Operating Travel: $300^\circ \pm 10^\circ$
 - Pulse Width Range: 500~2500 μs
 - Communication Mode: PWM
-

L298N – Dual H-Bridge Motor Driver:

Part Description

The L298N serves as the critical link between our RDK X5 logic controller and our 12V drive motor. Unlike our sensors which run off the 5V buck converter, the L298N is wired directly to the 11.1V battery line to provide maximum unchecked voltage to the drivetrain. We chose this specific H-Bridge because of its massive onboard heat sink and high power-dissipation rating (25W). Because we use a software "bit-bashing" control method rather than hardware PWM, the control signals sent to the driver's ENA pin can cause rapid switching states; the L298N's robust thermal design easily absorbs the resulting heat without failing during long competition rounds

Quick Spec

- Dual H-Bridge Driver
 - IC: L298N (ST),/li>
 - Logic voltage: 5VDC (has onboard 5V regulator if you would like to power from 12V)
 - Drive voltage: 5-35VDC
 - Drive current: 2A (MAX single bridge)
 - Dimensions: 43 x 57mm
-

DC Motor – 250 RPM Planetary

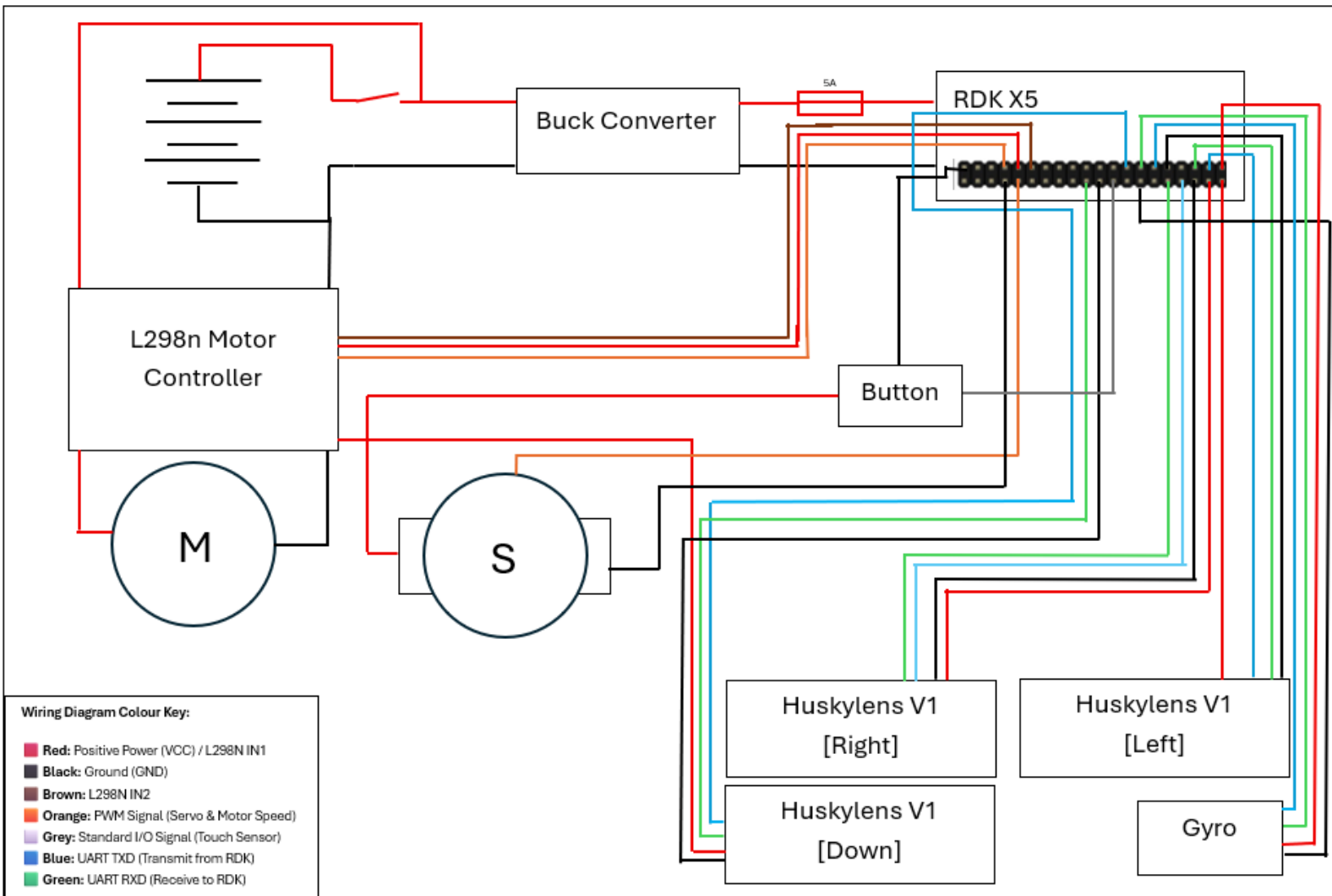
Part Description

This planetary DC motor powers our solid-axle rear-wheel drive. Selecting the right motor was one of our most important design trade-offs. We intentionally avoided high-speed motors in favour of this torque-oriented 250 RPM model. Because our RDK X5 utilizes bit-bashing for motor control, a highly geared, torque-heavy motor allows for much smoother and more predictable acceleration, eliminating the sudden "twitching" we experienced with faster motors. Additionally, its stall torque of 3.86 kg.cm easily overcomes the static friction and inertia of our fully loaded chassis, while the internal metal planetary gears prevent the transmission from stripping during emergency braking or rapid direction changes.

Quick Spec

- Operating Voltage: 12V (6-18V range)
 - Current (no load): 100mA
 - Current (load): 330mA
 - Stall Current: 1500mA
 - Speed (no load @ 12V): 250rpm
 - Speed (load @ 12V): 175rpm
 - Load Torque: 1.1kg.cm (0.107 Nm)
 - Stall Torque: 3.86kg.cm (0.378 Nm)
 - Gear Ratio: 1:35
 - Shaft Length: 8mm
 - Shaft Diameter: 4mm
 - Shaft Style: D-Shape
 - Dimensions: 52.6 x 24.4mm (diameter)
 - Weight: 87g
-

Wiring Diagram



RDX X5 Board 40Pin Table

Reuse function3	Reuse function2	Reuse function1	Reuse function 0	Function Description	X5 Pin Number	BCM Encoding	CVM Function	Physical Pin Board Encoding	CVM Function	BCM Encoding	X5 Pin Number	Function Description	Reuse function 0	Reuse function1	Reuse function2	Reuse function3
				3.3V power signal			VDD 3V3	1 2	VDD 5V			5V power signal				
	LSIO_GPIO0_11	SDA5	UART3_TXD	I2C0 data signal	387	2	I2C5 SDA	3 4	VDD 5V			5V power signal				
	LSIO_GPIO0_10	SCL5	UART3_RXD	I2C0 clock signal	389	3	I2C5 SCL	5 6	GND			GND signal				
		DSP_GPIO_09	DSP_MCLK1	I2S0 MCLK clock signal	420	4	I2S1 MCLK	7 8	UART TXD	14	383	UART1 send signal	UART1_TXD		LSIO_GPIO0_5	
				GND signal			GND	9 10	UART RXD	15	384	UART1 receive signal	UART1_RXD		LSIO_GPIO0_4	
	LSIO_GPIO0_1		UART7_TXD	GPIO17 signal	380	17	GPIO17	11 12	I2S1 BCLK	18	421	I2S1 BCLK clock signal	I2S1_BCLK	DSP_GPIO_10		
	LSIO_GPIO0_0		UART7_RXD	GPIO27 signal	379	27	GPIO27	13 14	GND			GND signal				
	LSIO_GPIO0_9		UART2_TXD	GPIO22 signal	388	22	GPIO22	15 16	GPIO23	23	382	GPIO23 signal	UART0_RTS	UART6_TXD	LSIO_GPIO0_3	
				3.3V power signal			VDD 3V3	17 18	GPIO24	24	402	GPIO24 signal	SPI2_MOSI	LSIO_GPIO0_23		LSIO_PWM_OUT3
	ITG_TDO	LSIO_GPIO0_19	SPI1_MOSI	SPI1 MOSI signal	398	10	SPI1_MOSI	19 20	GND			GND signal				
	ITG_TDI	LSIO_GPIO0_18	SPI1_MISO	SPI1 MISO signal	397	9	SPI1_MISO	21 22	GPIO25	25	387	GPIO25 signal	UART2_RXD		LSIO_GPIO0_8	
	ITG_TCK	LSIO_GPIO0_16	SPI1_SCLK	SPI1 CLK signal	395	11	SPI1_SCLK	23 24	SPI_CSNO	8	394	SPI1 SSN1 signal	SPI1_CSNO	LSIO_GPIO0_15	ITG_TMS	
				GND signal			GND	25 26	SPI_CSNO	7	396	SPI1 SSN0 signal	SPI1_CSNO	LSIO_GPIO0_17	ITG_TRSTN	
LSIO_PWM_OUT5		LSIO_GPIO1_8	SDA0	I2C3 clock signal	355	0	I2C0 SDA	27 28	I2C0_SCL	1	354	I2C0 signal	SCL0	LSIO_GPIO1_7		LSIO_PWM_OUT4
LSIO_PWM_OUT0		LSIO_GPIO0_20	SPI2_SCLK	GPIO5 signal	399	5	GPIO5	29 30	GND			GND signal				
LSIO_PWM_OUT1	TIME_SYNC2	LSIO_GPIO0_21	SPI2_SSNO	GPIO6 signal	400	6	GPIO6	31 32	PWM6	12	356	PWM6 signal	SCL1	LSIO_GPIO1_9	TIME_SYNC1	LSIO_PWM_OUT6
LSIO_PWM_OUT7		LSIO_GPIO1_10	SDA1	PWM0 signal	357	13	PWM7	33 34	GND			GND signal				
		DSP_GPIO_11	I2S1_LRCK	I2S0 LRCK signal	422	19	I2S1_LRCK	35 36	GPIO16	16	381	GPIO16 signal	UART0_CTS	UART6_RXD	LSIO_GPIO0_2	
LSIO_PWM_OUT2		LSIO_GPIO0_22	SPI2_MISO	GPIO26 signal	401	26	GPIO26	37 38	I2S1_SDIN	20	423	I2S1 DI signal	I2S1_DIN	DSP_GPIO_12		
				GND signal			GND	39 40	I2S1_SDOUT	21	424	I2S1 DO signal	I2S1_DOUT	DSP_GPIO_13		

Chassis/Other

Our chassis is primarily 3D-printed from Crealty ABS filament, manufactured by Prints by Paul. ABS was selected over PLA or PETG because of its thermal resistance and rigidity — ensuring the frame does not flex under load and can withstand the heat generated by the RDK X5 SBC and the L298N H-Bridge motor controller.

Material Choices

- **ABS (Chassis plates):**
 - High rigidity prevents flexing under torque.
 - Superior thermal tolerance compared to PLA, reducing risk of warping near heat-intensive components.
- **Metal (Motor brackets):**
 - Supplied with the starter kit.
 - Used where motors apply direct mechanical load, ensuring durability and preventing bracket deformation.

Structural Design

- **Two-plate layout:**
 - Plates separated by spacers for modularity and airflow.
 - Lower chassis: houses the servo motor, DC motor, and battery pack.
 - Upper chassis: houses the RDK X5, L298N motor controller, 5A buck converter, 3x Huskylens V1 camera, capacitive touch sensor (button), and DFRobot 6-axis gyro.
- **Rib reinforcement:**
 - Bottom chassis includes ribs to distribute stress and resist cracking.
 - Provides additional strength against torque forces from the DC motor and steering servo.
- **Airflow optimization:**
 - Designed with channels and spacing to direct airflow to thermal-intensive components.
 - Prevents overheating during extended runs.

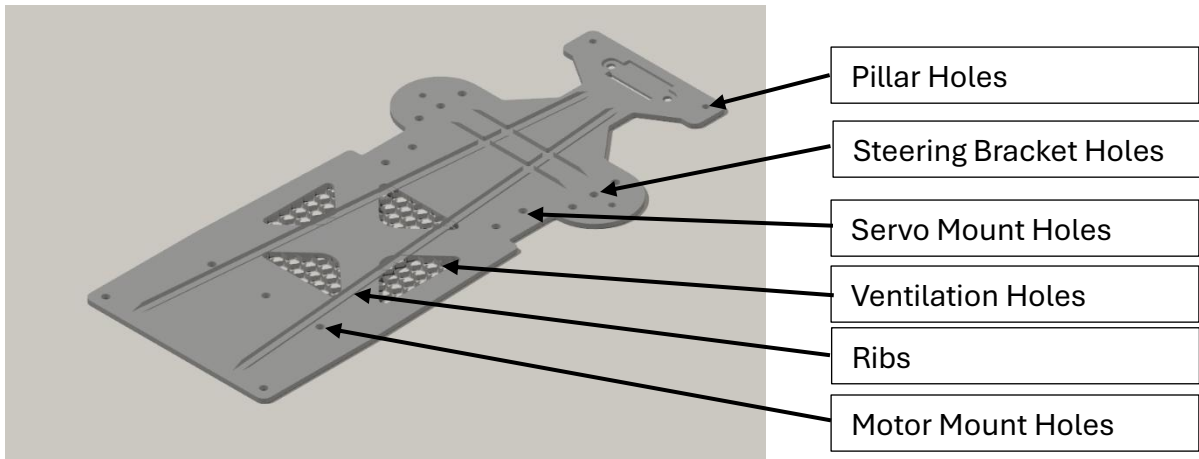
Drivetrain & Geometry:

- Configuration: Solid-axle rear-wheel drive, with front-wheel steering.
- Wheelbase: 115 mm (11.5 cm).
- Steering angle: Up to 50°, yielding a 40 cm turning radius.
- Wheel dimensions: 25 mm width, 65 mm diameter.
- Track width: 140 mm overall.

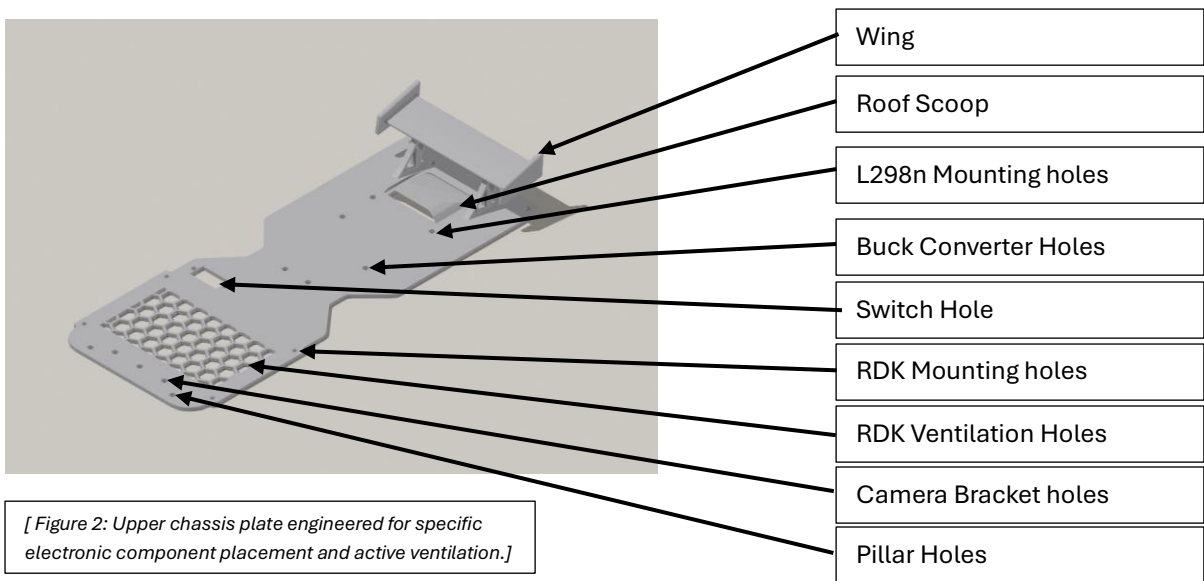
Functional & Aesthetic Features:

- Air scoop: positioned to “cool” the motor controller, while also adding a distinctive look.
- Wing handle: doubles as a carrying grip, especially useful during runaway test moments.
- Personal touches: break away from the typical cube-like or brick-like robot designs, giving the chassis personality and recognizability.

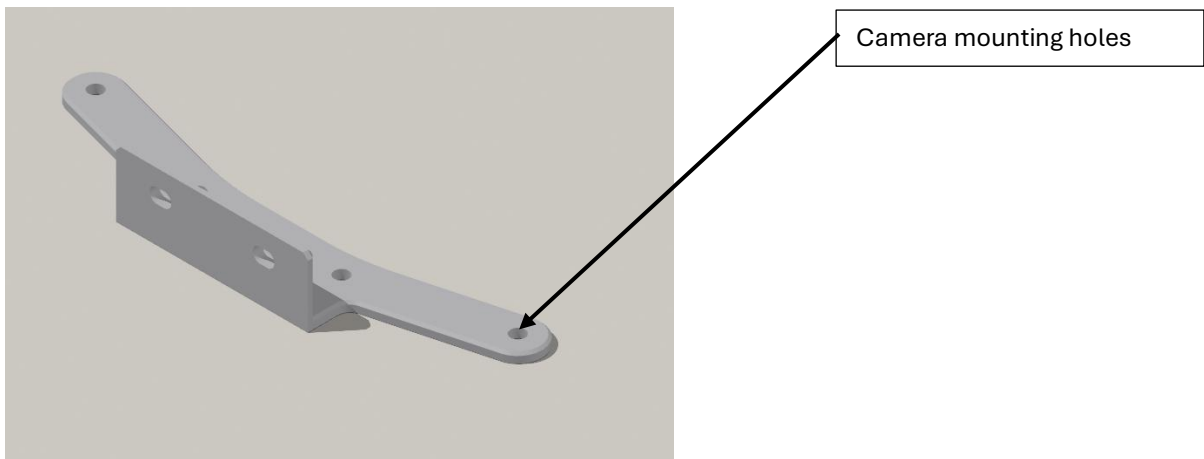
CAD Diagrams:



[Figure 1: Lower chassis plate highlighting longitudinal structural ribs and motor mounts.]



[Figure 2: Upper chassis plate engineered for specific electronic component placement and active ventilation.]



[Figure 3: Custom tri-camera HuskyLens mounting bracket.]

Software & Strategy

Round 1: Open Challenge Strategy

In Round 1, there are no traffic pillars, so our goal is maximum speed and efficiency.

1 - Orientation Logic:

The downward-facing camera scans the mat to identify the starting colour (blue or orange), determining if the robot must travel clockwise or counterclockwise.

2 - Heading & Execution:

The robot relies entirely on the DFRobot Gyro for its heading. It is programmed to hug the outer wall as closely as possible, allowing for a wider turning radius and maximizing our top speed while remaining stable.

Round 2: Obstacle Challenge Strategy

In Round 2, the robot must obey the red and green traffic pillars. To guarantee reliability, we split the logic between a "Mapping Lap" and "Execution Laps".

1 - Orientation Logic:

Similar to Round 1, the downward camera sets the initial orientation.

2. Lap 1 (Corner Mapping & Evaluation):

The robot drives forward using gyro heading. Crucially, the robot is programmed to stop at the start of a corner turn. It uses the forward-facing HuskyLens to evaluate the traffic pillar situated at the corner. Based on the pillar's colour and the robot's current orientation (CW/CCW), the algorithm determines if it needs to execute a "*narrow*" or "*sharp*" turn.

3 - Memory Storage:

The robot saves this specific corner turn type (narrow or sharp) into an array sequence.

4 - Laps 2 & 3 (Corner Execution & Real-Time Avoidance):

For the remaining two laps, the robot utilizes a hybrid approach:

Corner Turns (Memory): When the robot reaches the corners, it does not rely on the cameras. Instead, it reads the saved turn sequence (narrow or sharp) from memory and executes it "blindly." This is a vital engineering decision: by navigating corners purely from memory on Laps 2 and 3, the robot mitigates the risk of missing a pillar or executing a wrong turn due to minor offsets or imperfections that accumulate over the run.

Straightaway Pillars (Real-Time): For pillars located on the straight sides of the track, the robot actively uses the forward HuskyLens cameras to detect and avoid them in real-time, safely shifting lanes as necessary.

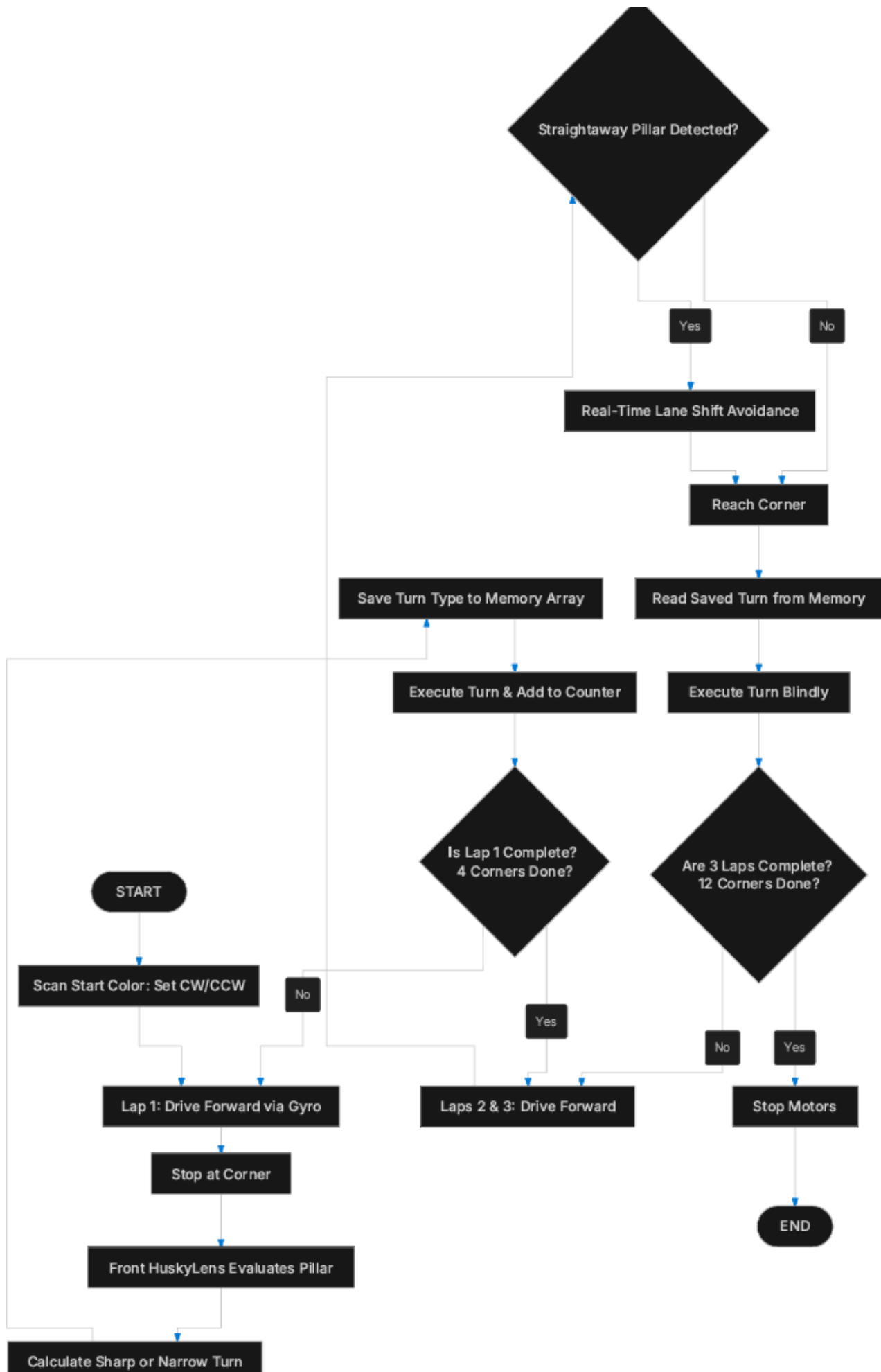
Calibration

Venue lighting is the biggest variable in computer vision. On the morning of the competition, we run a pre-flight check to verify if our HuskyLens AI cameras recognize the track elements under the new ambient lighting. If there are misreads, we execute a manual re-training protocol using the cameras' built-in physical 'Learn' button. All cameras operate in "Colour Recognition" mode. For the downward-facing camera, we clear the memory and train ID 1 for the Blue line and ID 2 for the Orange line. For the forward-facing cameras, we train multiple IDs per colour to account for shadows and glare: Green pillars are mapped to IDs 1, 2, and 6, while Red pillars are mapped to IDs 3, 4, and 5. Finally, the Magenta parking zone is mapped to ID 7.

Round 1 Flowchart



Round 2 Flowchart



Robot Journey

Changes from 2025 Season:

Main Controller

- **2025:** Raspberry Pi 4 4Gb
- **2026:** RDK X5 SBC (octa-core ARM Cortex-A55, 8 GB LPDDR4, 10 TOPS NPU, built-in Wi-Fi 6 + Bluetooth 5.4).
- **Reason for change:** Higher AI performance, modern connectivity, compact form factor, better GPIO support.

Vision System

- **2025:** Single Raspberry Pi Camera Module 3 (front-mounted, wide FOV).
- **2026:** Tri-camera HuskyLens AI setup (two forward-facing, one downward replacing colour sensor).
- **Reason for change:** Downward HuskyLens provides faster, more accurate mat colour/pattern recognition; forward HuskyLens improves obstacle and lane detection.

Sensors

- **2025:**
 - 5× VL53L0X time-of-flight sensors (front + sides).
 - TCS34725 colour sensor (underside).
- **2026:**
 - Huskylens V1 AI cameras (3 units).
 - DFRobot 6-axis gyro/accelerometer.
 - Capacitive touch sensor (button input).
- **Reason for change:** Simplified sensor suite, replaced colour sensor with downward HuskyLens for speed and accuracy, added gyro for orientation stability.

Movement & Steering

- **2025:** MG996R servo motor (front steering), 250 RPM planetary DC motor, L298N motor controller.
- **2026:** DS-R001 6 kg clutch servo (front steering), 250 RPM planetary DC motor, L298N motor controller.

- **Reason for change:** Clutch servo prevents burnout, reduces chassis stress, improves accuracy and lifespan.

Power System

- **2025:** 3S1P 18650 lithium pack (11.1 V nominal, 12.6 V max), XL4015 buck converter (5 V/5 A)
- **2026:** 3S1P 18650 lithium pack (11.1 V nominal, 12.6 V max), XL4015 buck converter (5 V/5 A), 5 A blade fuse + holder.
- **Reason for change:** Higher efficiency, modular pack rotation, improved protection.

Chassis & Geometry

- **2025:** Standard kit wheelbase, basic upper/lower chassis layout.
 - **2026:**
 - 3D-printed ABS chassis (Creality ABS, Prints by Paul).
 - Two-plate design with spacers, rib reinforcement for torque resistance.
 - Solid axle rear-wheel drive, front-wheel steering.
 - Wheelbase: 115 mm; Steering angle: 50°; Turning radius: 40 cm; Wheel width: 25 mm; Diameter: 65 mm; Track width: 140 mm.
 - Added airflow channels, air scoop and wing.
 - **Reason for change:** Improved thermal management, rigidity, and durability; added personality and usability features.
-

Monthly Logs:

WRO FE Monthly Log – January

(Research and Slow Start)

Repository & Documentation

- Created a GitHub repository to serve as a centralized backup and version control system for all project files throughout the year.

Power System Development

- Experimented with designing a custom power pack for the RDK X5.
- Ultimately selected a standard configuration: 3× 18650 lithium cells in series, mounted on the designated battery sled.
- Integrated a 5A 5V buck converter and a 5A inline blade fuse to ensure stable voltage regulation and protection for the RDK.

Software & Environment Setup

- Conducted research on ROS2 for potential integration.
- Set up the RDK X5 environment:
 - Learned to flash NAND using X Burn and to flash the RDK using RDK Studio.
 - Installed essential libraries and tools: matplotlib, numpy, smbus, smbus2, pip, git, VL53L0x, DFRobot I2C-Multiplexer, xfce, TCS34725, I2C-tools.

Hardware Research & Design

- 3D Printing Materials: Researched filament options and selected ABS for chassis printing due to durability and heat resistance.
- Camera Selection: Chose Huskylens V1 for vision capabilities.
- Chassis Improvements compared to 2025 robot:
 - Shortened wheelbase.
 - Lowered center of gravity by relocating battery pack to the bottom.
 - Added a separate layer for I2C lines.
 - Aligned time-of-flight sensors above wheels for improved responsiveness.

Thermal Management

- Investigated cooling solutions after observing RDK idle temperatures averaging 60°C, which could interfere with I2C performance.
- Considered adding heatsinks to improve thermal stability.

Progress Challenges

- Development pace slowed due to participation in FTC (FIRST Tech Challenge) during the month, which required time and focus alongside research activities.
-

WRO FE Monthly Log – February

(Component Arrival & Early Programming)

Component Acquisition

- Received key hardware parts: TP223 capacitive touch sensor, 5A blade fuse, and heatsinks for thermal management.

Programming & System Familiarization

- Began relearning programming basics on the RDK X5, noting differences in pin control compared to the Raspberry Pi 4.
- Focused on understanding GPIO handling and adapting workflows to the RDK's architecture.

Chassis Design

- Started designing the robot chassis with a three-layer structure:
 - Bottom layer: battery pack and power distribution.
 - Middle layer: sensor integration and wiring.
 - Upper layer: control board and camera mount.

Vision System Exploration

- Borrowed a Huskylens V1 from school.
- Learned basic usage, including how to load and run custom models for object tracking.

Component Preparation

- Organized and prepped available components for mounting once the chassis was printed.
- Ensured wiring and sensor placement plans were ready for rapid assembly.

Progress Challenges

- Experienced a two-week delay due to academic test series, temporarily slowing development.
-

WRO FE Monthly Log – March

(Chassis Printing & Setup Issues)

Component Acquisition

- Received VL53L1X Time-of-Flight sensor.
- Received ABS filament for chassis printing.

Printing & Fabrication

- Attempted to print the bottom and middle chassis on Mr. Olivier's printer (Bambulabs P1S) but encountered difficulties with ABS printing.
- Shifted to Paul (Prints by Paul), who used his Creality Ender 3 V2 to print the chassis successfully in ABS, ensuring reliable results.

Software & Environment Challenges

- Encountered issues with library installations, where several packages were installed to the wrong directories.
- Began troubleshooting installation paths to ensure proper integration with the RDK X5 environment.

Progress Challenges

- Development slowed during the first week of March due to academic test series.
 - Still waiting on Huskylens V1, which remains backordered in South Africa (estimated 4–6 week wait due to FTC demand).
-

WRO FE Monthly Log – April

(Code Re-Evaluation)

Programming Approach

- Took a deliberate step back to re-evaluate coding methodology for the RDK X5.
- Reflected on previous challenges with pin control and library integration, recognizing the need for a more structured and modular approach.
- Began reorganizing code into clearer modules and reusable templates, aiming for improved maintainability and easier troubleshooting.

Focus Areas

- Prioritized pin control consistency, ensuring mappings are documented and verified before implementation.
- Reviewed library installation paths to avoid misplacement issues experienced in March.
- Considered adopting a stepwise testing workflow, where each hardware component (sensors, motors, servos) is validated individually before integration into larger scripts.

Progress Challenges

- Slowed forward momentum due to the re-evaluation process, but this was intentional to prevent future inefficiencies.
 - Balanced robotics work with ongoing academic responsibilities.
-

WRO FE Monthly Log – May

(System Integration & Round 1 Coding)

Major Milestones

- May 2: Achieved full robot movement:
 - Steering system operational.
 - Motors successfully controlled.
 - Colour sensor integrated and functioning.
- May 18: Received and successfully set up the Huskylens V1 camera.
- May 23: Made a major design decision:
 - Removed the time-of-flight sensor layer entirely.
 - Transitioned to a simplified sensor suite: Huskylens camera, colour sensor, and an additional gyro sensor.

- May 26: Received, installed and configured the gyro sensor, ensuring stable readings and integration with the control system.
- May 27: Began developing the full Round 1 code and conducted initial testing.

Control Method – Bit-Bashing

- Continued using bit-bashing instead of PWM for servo steering and motor controller ENA pin control.
- Bit-bashing explained:
 - A software-driven method of manually toggling GPIO pins to simulate control signals.
 - Provides flexibility and direct control without relying on hardware PWM modules.
 - Requires precise timing loops in code to maintain accuracy and avoid drift.

Round 1 Code Plan

- Robot runs using only the gyro and colour sensor.
- Forward movement until detecting either blue or orange.
- Rotation logic:
 - Maintains an array of rotation values for orientation.
 - If blue is detected first → assigns counterclockwise array order.
 - If orange is detected first → assigns clockwise array order.
- Gyro sensor ensures straight-line stability.
- Colour sensor acts as the trigger for turns:
 - When the robot sees the same colour it detected at the start, it executes the corresponding turn.
- Testing results:
 - Right turns are smooth and stable.
 - Left turns show issues when synchronized with the gyro, requiring further refinement.

Mechanical Adjustments

- Adjusted the steering linkage for proper alignment.
 - Added a 2° toe-in angle on the wheels to improve stability and responsiveness.
-

WRO FE Monthly Log – June

(Exams & Structural Redesign)

Academic Commitments

- Progress slowed during June due to exam preparation and writing, limiting available time for robotics work.

Coding Progress – Round 1

- Continued refining the Round 1 code.
- Achieved semi-consistent first turns during testing.
- Identified that the gyro sensor required a 10° offset toward the inner wall:
 - This adjustment allowed the robot to drive slightly inward.
 - Provided a wider turning range, improving reliability of the first manoeuvre.

Servo Replacement – June 23

- The original MG996R servo burned out during testing.
- Replaced with the DFRobot 6 kg clutch servo
- Reason for switch:
 - Longer lifespan due to clutch mechanism preventing overload.
 - Protects chassis by slipping under excessive pressure instead of forcing the steering linkage.
 - Improved accuracy and stability, with less twitching compared to the MG996R.
- This change ensures smoother steering and reduces risk of mechanical damage.

Mechanical Challenges

- The original under chassis cracked during a test run.
- Highlighted the need for improved structural strength.

Redesign & Reinforcement

- Redesigned the under chassis to include ribs for structural support.
 - New design distributes stress more evenly, prevents cracking, and increases durability during repeated test cycles.
-

WRO FE Monthly Log – July

(Code Consistency & Hardware Adjustments)

July 1 – Code Consistency & Hardware Adjustments

- Achieved sufficient consistency in Round 1 code execution.
- Modified wheel gear ratios: switched from a speed-oriented ratio to a torque-oriented ratio for better control.
- Adjusted colour sensor positioning: added a 6 mm spacer to lower the sensor, resulting in more accurate readings.

July 2 – Refinement & Sensor Issue

- Focused on improving overall consistency of runs.
- Identified a gyro sensor drifting issue that required correction.

July 3 – Team Sync

- Held a team meeting to review progress and align on outstanding challenges.
- Discussed gyro drift and sensor calibration as next steps.

July 25 – OpenGL Course & Route Mapping

- Began an OpenGL course to integrate advanced visualization and mapping into the robot's navigation system.
 - Goal of OpenGL integration:
 - Use gyro + accelerometer data in combination with mat mapping.
 - Detect pillar positions and colour IDs using Huskylens.
 - Filter sensor data through an ID set, then through a route set.
 - Planned workflow:
 - Lap 1: Robot maps the board, calibrates speed, and identifies obstacle layout IDs on each side.
 - Lap 2 & 3: Robot replays Lap 1 route with improved speed and consistency.
-

WRO FE Monthly Log – August

(Regionals, Vision Upgrade & Speed Gains)

August – Regional Competition

- Competed in the regional round of WRO Future Engineers.
- Achieved 2nd place overall, demonstrating strong progress but also exposing areas for improvement.
- Round 1 code was functional but inconsistent, particularly struggling with navigation around extended walls.

Vision System Upgrade

- Decided to replace the colour sensor with a third HuskyLens AI camera facing downward.
- This change provided:
 - Higher accuracy in mat colour and feature detection.
 - Improved consistency in navigation, especially around extended walls.
 - Enabled the robot to take the outer wall route, reducing collisions.

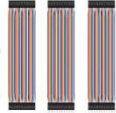
Performance Gains

- Speed increased from 17% to 80%, thanks to the HuskyLens upgrade.
- Practice lap times improved by 7 seconds per lap, a major step toward competitive consistency.

Mechanical Adaptation

- Designed and fabricated a custom bracket to mount all three HuskyLens cameras.
- The original two-camera mounting method was insufficient for the tri-camera setup.
- New bracket ensures stable alignment and optimal field of view for both forward and downward cameras.

Part Purchase page			
Part	Website	Photo	Count
RDK X5	Micro Robotics		X 1
18650 Batteries	Micro Robotics		X 15 *
Battery Sled	Micro Robotics		X 3 *
XT60 Connector	Micro Robotics		X 1
5A Fuse	Micro Robotics		X 1 (10 pack)
Fuse Holder	Micro Robotics		X 1
Buck Converter	Micro Robotics		X 1
HuskylensV1	Micro Robotics		X 3
Gyro	Micro Robotics		X 1
Touch Sensor	Micro Robotics		X 1 (4 pack)
300° 6Kg Servo	Micro Robotics		X 1
L298n	Micro Robotics		X 1
250 RPM DC 12V	Micro Robotics		X 1

ABS Filament	takealot.com		X 1
Dupont kit 20 cm	Micro Robotics		X 2
Dupont kit 10 cm	Micro Robotics		X 1
USB-C Male	Micro Robotics		X 1 (4 pack)
Nylon Spacer Kit	Micro Robotics		X1